

Block Cipher Modes of Operation: Galois/Counter Mode GCM

1 Definitions, Abbreviations, and Symbols

AAD	추가 인증 데이터(Additional Authenticated Data)
Additional Authenticated Data	인증 암호화 함수의 입력 데이터로, 인증 대상이지만 암호화하지 않는 데이터
Authenticated Decryption	암호문을 평문으로 복호화하고 암호문과 AAD의 진위성을 검증하는 GCM 함수
Authenticated Encryption	평문을 암호문으로 암호화하고 AAD와 암호문에 대한 인증 태그를 생성하는 GCM 함수
Authentication Tag (Tag)	우발적 오류와 의도적인 데이터 변경을 검출하도록 설계된 암호학적 검증값
Block	사용하는 블록 암호의 블록 크기와 같은 길이의 비트 문자열
Block Cipher	고정 길이 비트 문자열에 대한 순열의 집합이며, 키에 따라 순열이 결정됨
Block Size	주어진 블록 암호와 키에서 입력·출력 비트 문자열의 고정 길이
IV	초기화 벡터(Initialization Vector)
Nonce	지정된 문맥에서 한 번만 사용하는 값

2 GCM 개요 및 특징

GCM은 연관 데이터가 포함된 인증 암호화(AEAD) 방식으로, 데이터의 기밀성과 진위성·무결성을 동시에 제공하며 승인된 128비트 블록 암호(일반적으로 AES)와 함께 사용됨.

GCM은 일반 체크섬이나 오류 검출 코드보다 강한 인증 기능을 제공하며, 우발적 변경과 의도적인 무단 변경을 모두 검출할 수 있음.

GCM은 인증 암호화와 인증 복호화를 제공하며 병렬 처리가 가능해 하드웨어와 소프트웨어에서 높은 처리량을 구현할 수 있음.

2.1 주요 특징

- 기밀 데이터와 추가 인증 데이터의 길이를 미리 알지 못해도 순차적으로 처리할 수 있음.
- 기반 블록 암호의 암호화 방향 연산만 사용하며 복호화 방향 연산은 필요하지 않음.
- 암호문 복구와 별개로 보호 데이터의 진위성을 검증할 수 있음.
- IV와 데이터 길이를 알고 있으면 일부 블록 암호 연산을 미리 계산할 수 있음.
- 고정된 추가 인증 데이터는 관련 인증 연산을 미리 계산할 수 있음.

2.2 사용 시 주의사항

- 같은 키에서는 IV(Nonce)를 재사용하지 않음. 한 번이라도 재사용하면 인증 위조에 취약해질 수 있음.
- 호환성과 효율성을 위해 96비트 IV 사용을 권장함.
- 인증 태그는 일반적으로 96비트 이상 사용하며, 태그 길이 t 는 키와 함께 고정함.

3 수학적 구성 요소

3.1 비트 문자열 규칙

GCM의 입력과 중간값은 비트 문자열로 처리함. $\text{len}(X)$ 는 비트 길이, $\text{MSB}_s(X)$ 와 $\text{LSB}_s(X)$ 는 각각 왼쪽과 오른쪽 s 비트, $\parallel$ 는 연결, $\oplus$ 는 XOR을 의미함.

3.2 증가 함수

양의 정수 s 와 $\text{len}(X) \geq s$ 를 만족하는 비트 문자열 X 에 대해 s 비트 증가 함수 $\text{inc}_s(X)$ 를 정의함.

$$\text{inc}_s(X) = \text{MSB}_{\text{len}(X)-s}(X) \parallel [\text{int}(\text{LSB}_s(X)) + 1 \bmod 2^s]_s$$

- 카운터 모드에서 사용할 카운터의 하위 s 비트를 1씩 증가시키는 함수
- 오른쪽 끝의 s 비트(GCM에서는 보통 32비트)만 숫자로 취급해 1을 더하고($\bmod 2^s$), 나머지 상위 비트는 그대로 유지함.

3.3 블록 곱셈 연산(Galois 연산)

Algorithm 1: $X \bullet Y$

Input:
blocks X, Y .

Output:
block $X \bullet Y$.

Steps:

1. Let $x_0x_1\dots x_{127}$ denote the sequence of bits in X .
2. Let $Z_0 = 0^{128}$ and $V_0 = Y$.
3. For $i = 0$ to 127 , calculate blocks Z_{i+1} and V_{i+1} as follows:

$$Z_{i+1} = \begin{cases} Z_i & \text{if } x_i = 0; \\ Z_i \oplus V_i & \text{if } x_i = 1. \end{cases}$$

$$V_{i+1} = \begin{cases} V_i \gg 1 & \text{if } \text{LSB}_1(V_i) = 0; \\ (V_i \gg 1) \oplus R & \text{if } \text{LSB}_1(V_i) = 1. \end{cases}$$

4. Return Z_{128} .

- 입력: 128비트 블록 X, Y
- 출력: 128비트 곱셈 결과 블록 $X \cdot Y$

1단계: 초기값 설정 (Step 1~2)

- X 를 비트열 $x_0, x_1, \dots, x_{127}$ 로 표현함.
- 누적 결과 Z_0 를 0^{128} 로 초기화함.
- 작업 변수 V_0 를 Y 로 설정함.

2단계: 반복 계산 (Step 3)

$i = 0$ 부터 127 까지 다음 두 계산을 반복함.

누적 결과 Z 갱신

- $x_i = 0$ 이면 $Z_{i+1} = Z_i$ 로 유지함.
- $x_i = 1$ 이면 $Z_{i+1} = Z_i \oplus V_i$ 로 갱신함.

작업 변수 V 갱신

- $\text{LSB}_1(V_i) = 0$ 이면 $V_{i+1} = V_i \gg 1$ 로 설정함.
- $\text{LSB}_1(V_i) = 1$ 이면 $V_{i+1} = (V_i \gg 1) \oplus R$ 로 설정함.

- R은 $11100001 \parallel 0^{120}$ 으로 정의되며 유한체 곱셈의 모듈러 축소에 사용함.
- 이 과정은 $GF(2^{128})$ 에서 환원 다항식 $x^{128} + x^7 + x^2 + x + 1$ 을 적용하는 연산임.

3단계: 최종 결과 반환 (Step 4)

128회 반복 후 Z_{128} 을 $X \cdot Y$ 의 결과로 반환함.

3.4 GHASH 함수

Algorithm 2: $GHASH_H(X)$

Prerequisites:

block H , the hash subkey.

Input:

bit string X such that $\text{len}(X) = 128m$ for some positive integer m .

Output:

block $GHASH_H(X)$.

Steps:

1. Let $X_1, X_2, \dots, X_{m-1}, X_m$ denote the unique sequence of blocks such that $X = X_1 \parallel X_2 \parallel \dots \parallel X_{m-1} \parallel X_m$.
2. Let Y_0 be the “zero block,” 0^{128} .
3. For $i = 1, \dots, m$, let $Y_i = (Y_{i-1} \oplus X_i) \cdot H$.
4. Return Y_m .

- 준비물(H): 해시 전용 서브키(Hash Subkey)로, 암호화 키 K로 0^{128} 블록을 암호화해 미리 생성하는 128비트 값임.
- 입력(X): 인증할 전체 데이터로, GHASH 입력 길이는 128비트의 배수여야 하며 부족한 길이는 4절의 GCM 알고리즘에서 0으로 채움.
- 출력(Y_m): 전체 데이터를 128비트로 압축한 중간 블록임.

1단계: 128비트씩 쪼개기 (Step 1)

입력 X를 128비트 단위로 분할함.

분할된 블록을 순서대로 $X_1, X_2, \dots, X_m$ 으로 정의함.

2단계: 초기값 준비 (Step 2)

연쇄 계산의 초기값 Y_0 를 0^{128} 로 설정함.

3단계: 연쇄 압축 (Step 3)

$$Y_i = (Y_{i-1} \oplus X_i) \cdot H$$

GHASH의 핵심 연산으로, 1번 블록부터 m 번 블록까지 다음 계산을 반복함.

더하기($\oplus$): 이전 단계의 결과 Y_{i-1} 과 현재 데이터 블록 X_i 를 XOR함.

곱하기($\cdot H$): XOR 결과에 해시 서브키 H 를 곱하며, 일반적인 정수 곱셈이 아닌 $GF(2^{128})$ 의 유한체 곱셈을 사용함.

X_1 부터 각 블록을 순서대로 XOR하고 H 를 곱해 $Y_1, Y_2, \dots, Y_m$ 을 연쇄적으로 생성함.

4단계: 중간 블록 출력 (Step 4)

마지막 블록 X_m 까지 연산한 결과 Y_m 을 반환함. 이 값은 최종 인증 태그가 아니라 태그 생성에 사용되는 중간 블록임.

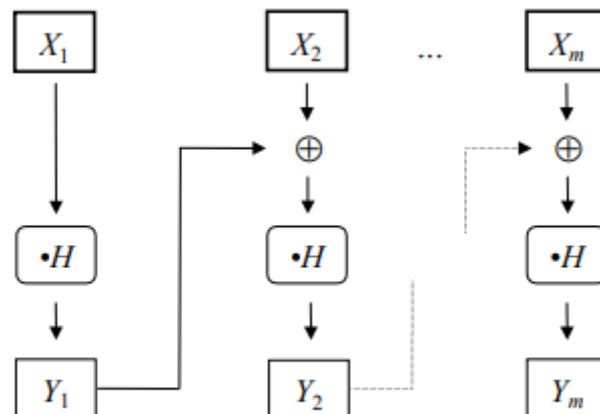


Figure 1: $\text{GHASH}_H(X_1 \parallel X_2 \parallel \dots \parallel X_m) = Y_m$.

3.5 GCTR 함수(CTR)

$$128\text{-bit Counter Block} = \underbrace{[96\text{-bit Nonce / IV}]}_{\text{고정 (변하지 않음)}} \parallel \underbrace{[32\text{-bit Counter}]}_{\text{inc}_{32} \text{ 연산 대상}}$$

Algorithm 3: $GCTR_K(ICB, X)$

Prerequisites:

approved block cipher CIPH with a 128-bit block size;
key K .

Input:

initial counter block ICB ;
bit string X , of arbitrary length.

Output:

bit string Y of bit length $\text{len}(X)$.

Steps:

1. If X is the empty string, then return the empty string as Y .
2. Let $n = \lceil \text{len}(X)/128 \rceil$.
3. Let $X_1, X_2, \dots, X_{n-1}, X_n^*$ denote the unique sequence of bit strings such that
$$X = X_1 \parallel X_2 \parallel \dots \parallel X_{n-1} \parallel X_n^* ;$$
$$X_1, X_2, \dots, X_{n-1} \text{ are complete blocks.}^2$$
4. Let $CB_1 = ICB$.
5. For $i = 2$ to n , let $CB_i = \text{inc}_{32}(CB_{i-1})$.
6. For $i = 1$ to $n-1$, let $Y_i = X_i \oplus \text{CIPH}_K(CB_i)$.
7. Let $Y_n^* = X_n^* \oplus \text{MSB}_{\text{len}(X_n^*)}(\text{CIPH}_K(CB_n))$.
8. Let $Y = Y_1 \parallel Y_2 \parallel \dots \parallel Y_n^*$.
9. Return Y .

- 입력(Input): ICB(초기 카운터 블록), X(암호화하거나 복호화할 원본 데이터)
- 출력(Output): Y(처리가 완료된 데이터로, 입력 X와 길이가 같음)

1단계: 데이터 분할 (Step 1~3)

입력 X 를 128비트(16바이트) 단위로 분할함.

완전한 블록을 $X_1, X_2, \dots, X_{n-1}$ 로 정의함.

입력 길이가 128비트의 배수가 아니면 마지막 부분 블록을 X_n^* 으로 정의하며, 배수이면 마지막 블록도 완전한 128비트 블록임.

2단계: 카운터 블록 생성 (Step 4~5)

초기 카운터 ICB를 첫 번째 카운터 CB_1 으로 사용함.

두 번째 카운터부터 이전 카운터의 오른쪽 32비트만 1씩 증가시키는 inc_{32} 함수를 사용해 $CB_2, CB_3, \dots, CB_n$ 을 생성함.

3단계: 완전한 블록 처리 (Step 6)

$$Y_i = X_i \oplus \text{CIPH}_K(\text{CB}_i)$$

카운터 CB_i 를 AES와 같은 블록 암호 CIPH_K 로 처리해 128비트 키스트림 블록을 생성함.

키스트림 블록과 128비트 입력 블록 X_i 를 XOR하여 출력 블록 Y_i 를 생성함.

4단계: 부분 블록 처리 (Step 7)

$$Y_n^* = X_n^* \oplus \text{MSB_len}(X_n^*)(\text{CIPH}_K(\text{CB}_n))$$

마지막 부분 블록 X_n^* 이 40비트인 경우, CB_n 의 128비트 키스트림 중 앞의 40비트만 MSB 함수로 잘라 사용함.

잘라낸 40비트 키스트림과 40비트 입력을 XOR하여 같은 길이의 출력 Y_n^* 을 생성함. 따라서 별도의 패딩은 필요하지 않음.

5단계: 최종 연결 (Step 8~9)

처리된 블록을 $Y = Y_1 \parallel Y_2 \parallel \dots \parallel Y_n^*$ 순서로 연결해 최종 출력 Y 를 반환함.

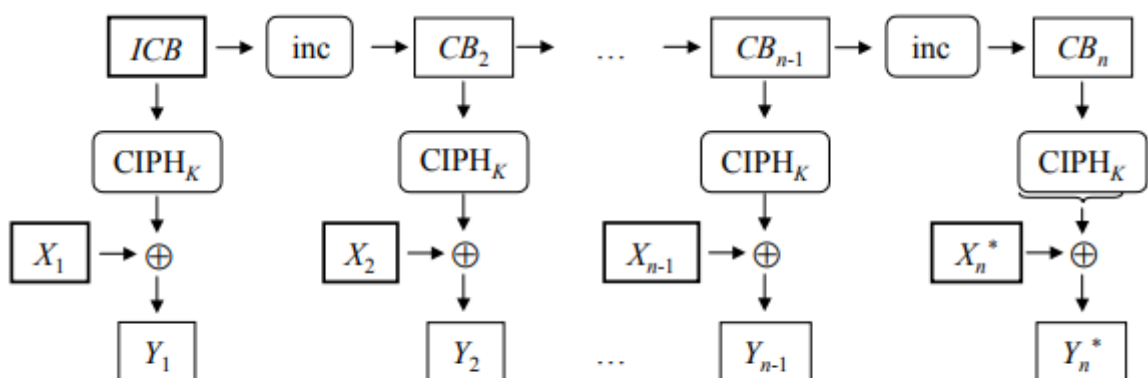


Figure 2: $\text{GCTR}_K(\text{ICB}, X_1 \parallel X_2 \parallel \dots \parallel X_n^*) = Y_1 \parallel Y_2 \parallel \dots \parallel Y_n^*$.

4 GCM 규격

4.1 인증 암호화 알고리즘

Algorithm 4: GCM-AE_K(IV, P, A)

Prerequisites:

approved block cipher CIPH with a 128-bit block size;
key K ;
definitions of supported input-output lengths;
supported tag length t associated with the key.

Input:

initialization vector IV (whose length is supported);
plaintext P (whose length is supported);
additional authenticated data A (whose length is supported).

Output:

ciphertext C ;
authentication tag T .

Steps:

1. Let $H = \text{CIPH}_K(0^{128})$.
2. Define a block, J_0 , as follows:
If $\text{len}(IV)=96$, then let $J_0 = IV \parallel 0^{31} \parallel 1$.
If $\text{len}(IV) \neq 96$, then let $s = 128 \lceil \text{len}(IV)/128 \rceil - \text{len}(IV)$, and let
 $J_0 = \text{GHASH}_H(IV \parallel 0^{s+64} \parallel [\text{len}(IV)]_{64})$.
3. Let $C = \text{GCTR}_K(\text{inc}_{32}(J_0), P)$.
4. Let $u = 128 \cdot \lceil \text{len}(C)/128 \rceil - \text{len}(C)$ and let $v = 128 \cdot \lceil \text{len}(A)/128 \rceil - \text{len}(A)$.
5. Define a block, S , as follows:
 $S = \text{GHASH}_H(A \parallel 0^v \parallel C \parallel 0^u \parallel [\text{len}(A)]_{64} \parallel [\text{len}(C)]_{64})$.
6. Let $T = \text{MSB}_t(\text{GCTR}_K(J_0, S))$.
7. Return (C, T) .

평문 P 와 추가 인증 데이터 A 를 입력받아 암호문 C 와 인증 태그 T 를 생성함.

- **입력:** IV(초기화 벡터), P(평문), A(추가 인증 데이터 AAD)
- **출력:** C(암호문), T(인증 태그)
- **1단계: 해시 키 생성**
0으로 채운 128비트 블록 0^{128} 을 블록 암호로 암호화해 GHASH에서 사용할 해시 서브 키 H 를 생성함.
- **2단계: 초기 카운터 블록 준비**
IV를 규격에 맞는 128비트 Pre-counter block J_0 로 가공함.

- IV가 96비트이면 $J_0 = IV \parallel 0^{31} \parallel 1$ 로 구성함.
- IV가 96비트가 아니면 GHASH를 사용해 128비트 J_0 를 생성함.
- 3단계: 평문 암호화
 $\text{inc}_{32}(J_0)$ 를 첫 카운터로 사용해 평문 P 를 GCTR로 처리하고 암호문 C 를 생성함.
- 4·5단계: 인증 데이터 압축
 A , 암호문 C , 패딩 및 두 데이터의 길이를 연결한 뒤 GHASH에 입력해 128비트 블록 S 를 생성함.
- 6단계: 인증 태그 생성
 S 를 J_0 와 함께 GCTR에 입력하고, 결과의 앞 t 비트(MSB_t)를 최종 인증 태그 T 로 사용함.
- 7단계: 최종 출력
 암호문 C 와 인증 태그 T 를 반환함.

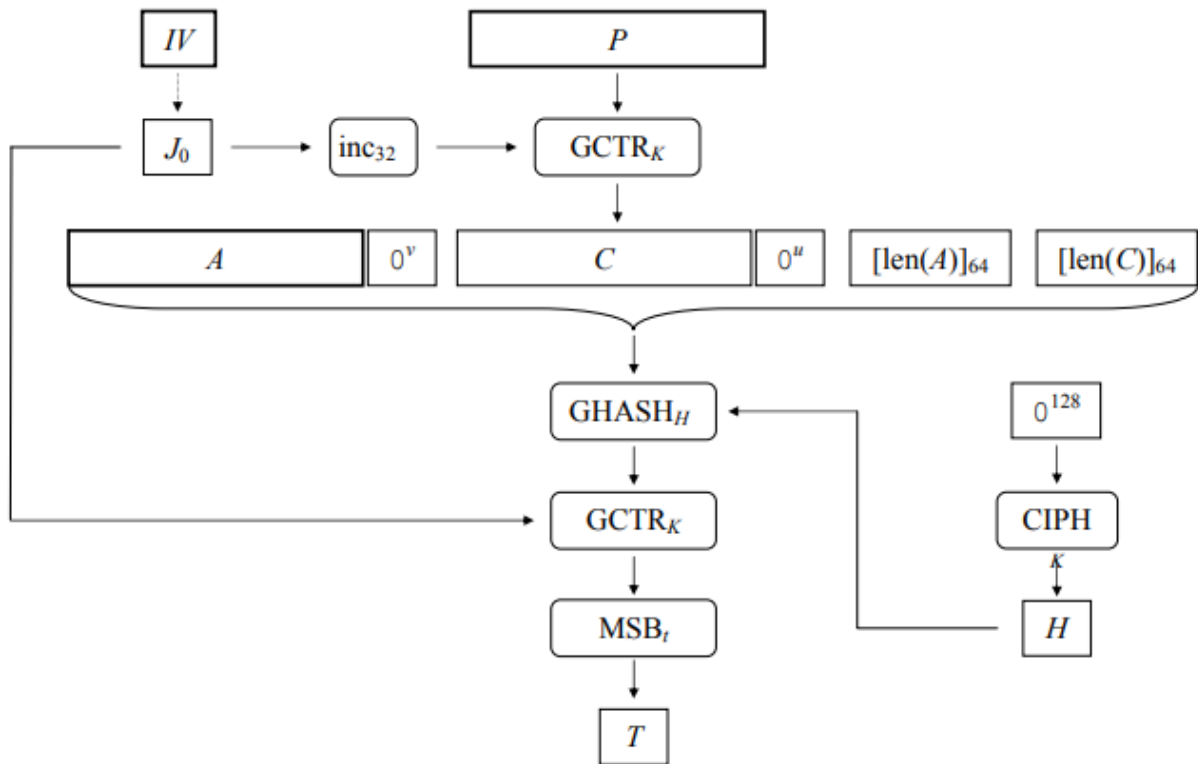


Figure 3: $\text{GCM-AE}_K(IV, P, A) = (C, T)$.

4.2 인증 복호화 알고리즘

Algorithm 5: GCM-AD_K(IV, C, A, T)

Prerequisites:

approved block cipher CIPH with a 128-bit block size;
key K ;
definitions of supported input-output lengths;
supported tag length t associated with the key.

Input:

initialization vector IV ;
ciphertext C ;
additional authenticated data A ;
authentication tag T .

Output:

plaintext P or indication of inauthenticity *FAIL*.

Steps:

1. If the bit lengths of IV , A or C are not supported, or if $\text{len}(T) \neq t$, then return *FAIL*.
2. Let $H = \text{CIPH}_K(0^{128})$.
3. Define a block, J_0 , as follows:
If $\text{len}(IV)=96$, then $J_0 = IV \parallel 0^{31} \parallel 1$.
If $\text{len}(IV) \neq 96$, then let $s = 128 \lceil \text{len}(IV)/128 \rceil - \text{len}(IV)$, and
 $J_0 = \text{GHASH}_H(IV \parallel 0^{s+64} \parallel [\text{len}(IV)]_{64})$.
4. Let $P = \text{GCTR}_K(\text{inc}_{32}(J_0), C)$.
5. Let $u = 128 \cdot \lceil \text{len}(C)/128 \rceil - \text{len}(C)$ and let $v = 128 \cdot \lceil \text{len}(A)/128 \rceil - \text{len}(A)$.
6. Define a block, S , as follows:
 $S = \text{GHASH}_H(A \parallel 0^v \parallel C \parallel 0^u \parallel [\text{len}(A)]_{64} \parallel [\text{len}(C)]_{64})$
7. Let $T' = \text{MSB}_t(\text{GCTR}_K(J_0, S))$.
8. If $T = T'$, then return P ; else return *FAIL*.

1단계: 입력 규격 검사

IV, A, C의 길이와 수신 태그 T의 길이를 확인하고, 지원 범위를 벗어나면 즉시 FAIL을 반환함.

2.3단계: 해시 키와 초기 카운터 준비

암호화 과정과 동일하게 해시 서브키 H를 생성하고 IV로부터 초기 카운터 블록 J_0 를 재구성함.

4단계: 평문 복구

$\text{inc}_{32}(J_0)$ 를 첫 카운터로 사용해 암호문 C를 GCTR로 처리하고 평문 후보 P를 복구함. 태그 검

증 전에는 이 평문을 사용하지 않음.

5.6단계: 수신 데이터 압축

수신한 A와 C, 패딩 및 길이 정보를 GHASH에 입력해 128비트 블록 S를 생성함.

7단계: 검증용 태그 생성

$GCTR_K(J_0, S)$ 를 계산하고 결과의 왼쪽 t비트(MSB_t)를 잘라 검증용 인증 태그 T'를 생성함.

8단계: 태그 비교 및 최종 결정

- 수신 태그 T와 새로 계산한 태그 T'를 일정 시간 비교 방식으로 검증함.
- 두 태그가 일치하면 복구한 평문 P를 반환함.
- 두 태그가 다르면 평문을 폐기하고 FAIL만 반환함.

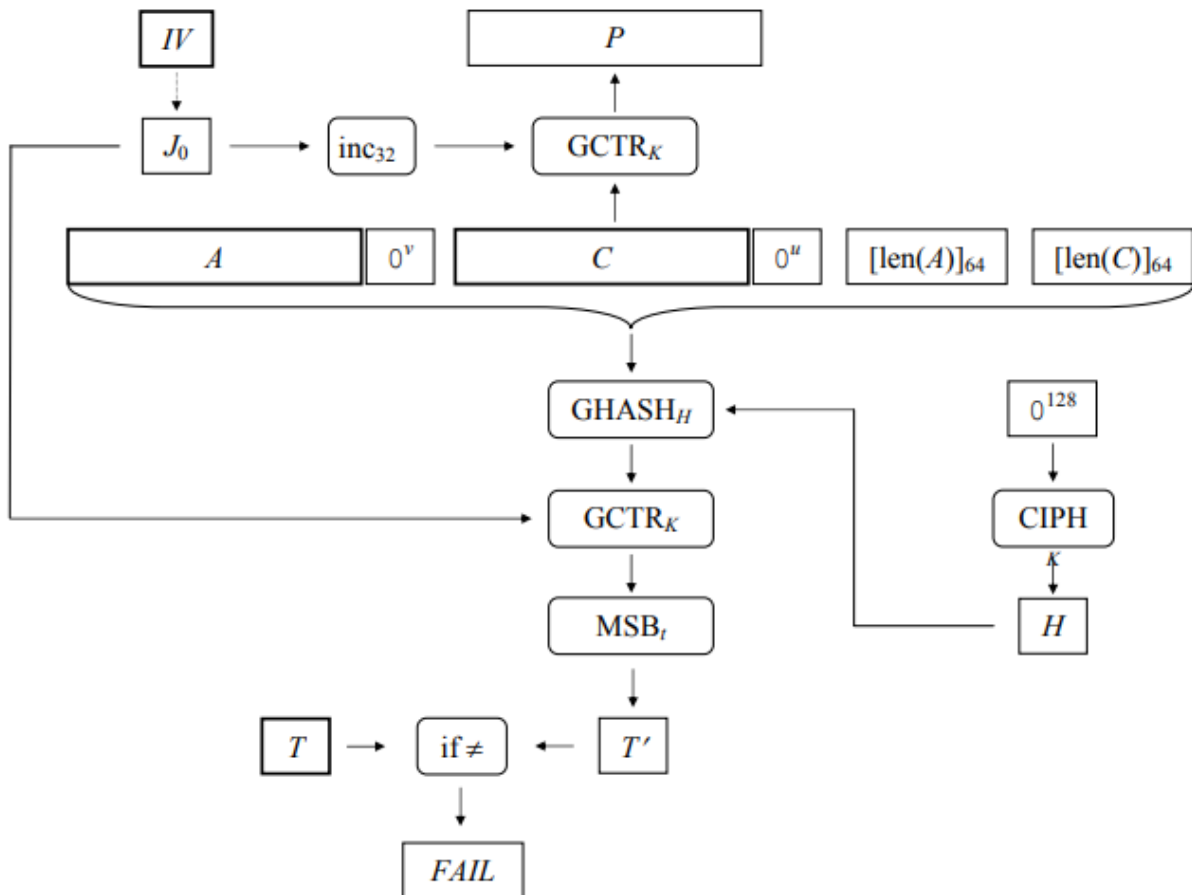


Figure 4: $GCM-AD_K(IV, C, A, T) = P$ or $FAIL$.